

# Start

Tim Schmitz  
21.02.2024



## Inhalt

|                           |   |
|---------------------------|---|
| Interpreter .....         | 3 |
| Übersetzungsprozess ..... | 3 |
| Interaktion .....         | 4 |
| Modularisierung .....     | 4 |
| Attribute .....           | 5 |

# Start

Um mit der Programmierung mit Python zu starten, benötigt man zunächst einen Interpreter sowie einen Editor. Die Beispiele wurden ausgeführt auf der Linux-Applikation Ubuntu 18.04 LTS unter Windows 10.

## Interpreter

Der Interpreter ist eine Software, die ein Python-Programm ausführt. Auf vielen Betriebssystemen wie etwa Linux ist er vorinstalliert. Der Interpreter kann auf zwei Arten aufgerufen werden:

1. Als Programm, das ein vorgegebenes Skript ausführt
2. Als Kommandozeilen-Werkzeug, das hintereinander Kommandos ausführt

Die folgenden Beispiele wurden alle mit der Entwicklungsumgebung vim (Visual Editor Improved) und dem Python 3.6.7-Interpreter für Linux ausgeführt, insbesondere wenn die Syntax im Header an den Aufruf des Linux-Interpreters erinnert. Das folgende Programm führt zwei Ausgaben auf der Kommandozeile aus:

```
#!/usr/bin/python3
# helloWorld.py

print('hello world')
print(2 ** 100)
```

Gestartet wird das Programm mit dem folgenden Befehl `python` auf der Kommandozeile. Der Befehl startet den Interpreter und liefert als Ergebnis die Ausgabe eines Strings und einer Zahl, genauer gesagt die einer arithmetischen Operation (2 hoch 100).

```
# python helloWorld.py
hello world
1267650600228229401496703205376
```

## Übersetzungsprozess

Der Übersetzungsprozess einer Python-Datei ist denkbar einfach und ähnelt denen der anderer Skriptsprachen. Er ist in folgender Abbildung dargestellt:

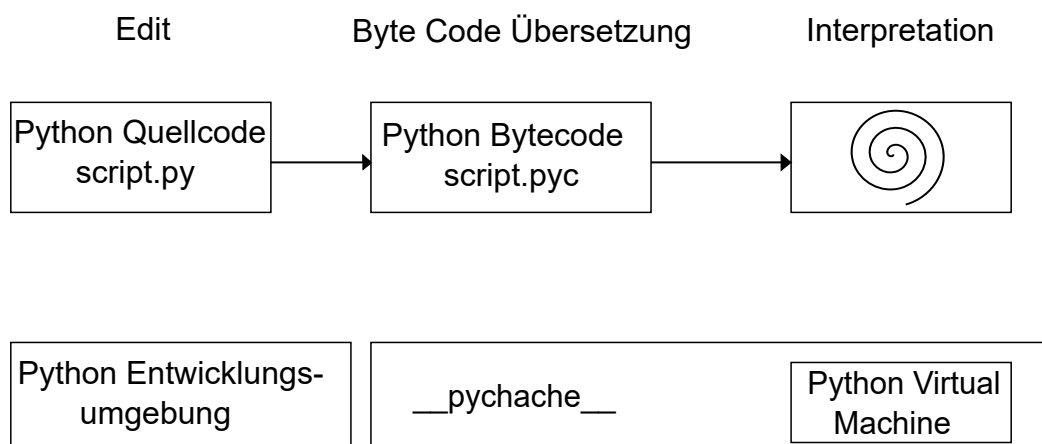


Abbildung 1: Übersetzung einer Python-Datei

Nachdem die Datei editiert wurde, wird sie über den Python-Interpreter übersetzt, indem der Python-Interpreter ausgeführt wird. Dabei wird im ersten Schritt durch Python der Quellcode in den sogenannten Byte-Code übersetzt. Die Kompilation ist hier lediglich eine Übersetzung einer Hochsprache in eine niedrigere, plattform-unabhängige Repräsentation des Quellcodes. Dabei ist die Dekomposition in Byte-Code nützlich, da dieser Code sehr viel schneller als der Quellcode interpretiert werden kann.

Die Übersetzung in Byte-Code bleibt normalerweise dem Ausführenden verborgen. Falls der Python-Prozess allerdings Zugriffsrechte auf den lokalen Rechner besitzt, dann speichert dieser den Byte-Code in einer Datei mit dem Suffix `.pyc` (python compiled). Diese Dateien werden dann in einem

Verzeichnis mit dem Namen `__pycache__` gespeichert, welches in dem gleichen Verzeichnis wie das der Quelldateien angelegt wird. Hierbei wird auch die Python-Version referenziert.

Anschließend wird die Byte-Code-Datei an die Python Virtual Machine (PVM) ausgeliefert, die als Laufzeitumgebung iterativ den Byte-Code abarbeitet. Technisch gesehen ist dies nur der letzte Schritt der Interpretation einer Python-Datei.

Abgesehen von impliziten Nachteilen in der Performance gegenüber binär übersetzten Sprachen bietet das Entwicklungsmodell den Vorteil, dass hier zwischen Produktions- und Entwicklungsumgebung nicht mehr unterschieden werden kann.

## Interaktion

Es gibt ebenfalls die Möglichkeit, den Interpreter interaktiv (über den Prompt) aufzurufen. Dies geschieht auf der Linux-Konsole wie folgt:

```
# python
Python 3.6.7 (default, Oct 22 2018, 11:32:17)
[GCC 8.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> CTRL + D
```

Der Vorteil des interaktiven Prompts liegt darin, sehr schnell Kommandos ausführen zu können. Zum Beispiel muss man bei der Ausgabe von Variablen nicht das Schlüsselwort `print` angeben, wenn man den Wert einer zuvor zugewiesenen Variablen ausgeben will. Fehler hingegen werden nicht zugelassen, wie zum Beispiel der Gebrauch nicht-initialisierter Variablen. Es ist auch möglich, komplexere Ausdrücke im interaktiven Prompt auszuwerten wie in folgendem Beispiel:

```
>>> import os
>>> os.getcwd()
'/home/tim/python/start'
```

Zunächst wird vom Interpreter ein vorkodiertes Modul importiert. Anschließend wird eine Standardbibliothek ausgegeben. Sie meldet das aktuelle Arbeitsverzeichnis (get current working directory), das direkt vom Prompt ausgegeben wird. Statements, die sich über mehrere Zeilen erstrecken, sollten daher besser in Form von Skripten ausgeführt werden.

## Modularisierung

Wenn man Programme als Sequenzen interpretierbarer Statements auffasst, dann werden sie in Dateien gespeichert, die als Module bezeichnet werden. Es sind einfache Text-Dateien, die entweder direkt vom Interpreter ausgeführt werden können oder die von anderen Programmen, den Hauptprogrammen oder Skripten, geladen und ausgeführt werden. Die Beziehungen sind in der folgenden Abbildung zu sehen:

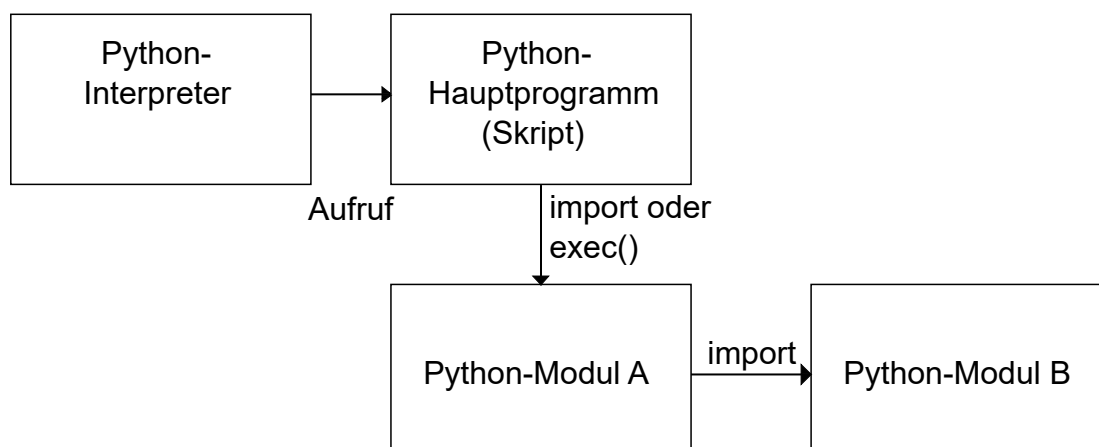


Abbildung 2: Modularisierung von Python-Programmen

Das Modul, was nun von verschiedenen anderen Programmen ausgeführt werden soll, lautet wie folgt:

```
#!/usr/bin/python
# module.py
```

```
#-----#
# Laden und Ausgabe eines Bibliotheksmoduls #
#-----#
import sys
print(sys.platform) # aktuelles Betriebssystem

#-----#
# Ausgabe einer Zahl und einer String-Wiederholung #
#-----#
print(2 ** 32) # 2 hoch 32
x = 'Spam!'
print(x * 8) # achtfache Wiederholung von 'Spam!'
```

Natürlich könnte man das Modul auch direkt mit dem Interpreter via `python module.py` ausführen. Hier soll aber ein Ansatz betrachtet werden, der die Programm-Architektur von Python in den Mittelpunkt stellt. Dabei lädt ein Hauptprogramm (das Skript) das Modul und der Import des Moduls ist das Ausführen des Codes. Die folgende Sequenz des Hauptprogramms `importModule.py` importiert das Modul und führt den Code aus:

```
#!/usr/bin/python3
# moduleImport.py

import module
```

Das Ergebnis auf der Kommandozeile lautet wie folgt:

```
# python moduleImport.py
linux
4294967296
Spam!Spam!Spam!Spam!Spam!Spam!Spam!Spam!
```

## Attribute

Grundsätzlich gilt: ein Modul ist eigentlich ein Paket von Variablennamen, der sogenannte Namensraum oder Namespace. Die Namen dieses Pakets werden Attribute genannt. Ein Attribut ist also nichts anderes als ein Variablennamen, der an einem bestimmten Objekt (wie etwa einem Modul) hängt.

Der Zugriff auf Attribute von Modulen geschieht in Python über die Statements `import` und `from` (auch im Zusammenhang mit dem `reload`-Statement). Beispielsweise beinhaltet das folgende Modul `file.py` ein Attribut `title`:

```
#!/usr/bin/python3
# file.py

title = "The Meaning of Life"
```

Der Zugriff erfolgt über einen dezidierten und qualifizierten Import:

```
#!/usr/bin/python3
# fileTitleImport.py

import file
print(file.title)
```

Das Ergebnis auf der Kommandozeile lautet dann:

```
# python fileTitleImport.py
The Meaning of Life
```

Alternativ zu dem Zugriff auf Objekt-Attribute über das `Import`-Statement kann über das `from`-Statement Namen von Modulen abgreifen (tatsächlich werden sie kopiert):

```
#!/usr/bin/python3
# fileTitleImportFrom.py

from file import title
print(title)
```

Die Built in-Funktion `dir` kann dagegen noch viel mehr liefern als lediglich die Objekt-Attribute, sondern alle Namen, die innerhalb eines Moduls verfügbar sind. Dazu dient folgenden Import-Sequenz:

```
#!/usr/bin/python3
# fileTitleImportDir.py

import file
print(dir(file))
```

Das Ergebnis liefert folgende Liste zurück:

```
# python fileTitleImportDir.py
['__builtins__', '__cached__', '__doc__', '__file__', '__loader__', '__name__', '__package__', '__spec__', 'title']
```

Alle Namen, die in diesem Ergebnisarray als doppelt unterstrichen gekennzeichnet sind, sind eingebaute Built In-Namen, die von Python vordefiniert sind und eine spezielle Bedeutung für den Interpreter haben.

Eine weitere Alternative zum Import-Statement stellt das Exec-Statement dar, das dazu dient, den Programmcode eines gerufenen Moduls zu durchlaufen. Dabei wird das gerufene Modul nicht geladen. Vorsicht ist geboten beim Aufruf der Exec-Funktion, denn lokale Variablen können durch den Aufruf von Modulvariablen überschrieben werden. Dies kann im Import nicht geschehen, da dieser die Namespaces respektiert. Der Exec-Aufruf hat folgende Struktur:

```
#!/usr/bin/python3
# moduleExec.py

exec(open('module.py').read())
```

Das Ergebnis ist das gleiche wie bei dem Import.